第一章 数据库绪论

数据（data）是数据库中存储的基本对象
数据库（database，简称 DB）是长期储存在计算机内有组织、可共享的大量数据的集合。
数据库管理系统位于用户与操作系统之间的一层数据管理软件
数据库系统的构成

- 数据库
- 数据库管理系统（及外围的应用开发工具）
- 应用程序
- 数据库管理员

数据库管理系统的主要功能

- **①数据定义功能**
- **②数据组织、存储和管理功能**
- **③数据操纵功能**
- **④数据库的事务管理和运行管理功能**
- **⑤数据库的建立和维护功能功能**
- **⑥其他功能功能**

DBMS 的 DCL：安全性保护、完整性检查、并发控制、数据库恢复

表 1.1 数据管理技术三个阶段的比较

对比项	人工管理阶段	文件系统阶段	数据库系统阶段
应用领域	科学计算	科学计算、数据管理	大规模数据管理
主要硬件	无直接存取存储设备	磁盘、磁鼓	大容量磁盘、磁盘阵列
主要软件	没有操作系统，没有管理数据的专门软件	有文件系统	有数据库管理系统
处理方式	批处理	联机实时处理、批处理	联机实时处理、分布处理、批处理
数据的管理者	人(程序员)	文件系统	数据库管理系统
数据面向的对象	某一应用程序	某一应用	现实世界(部门、企业、跨国组织等)
数据的共享程度	不共享，冗余度极大	共享性弱，冗余度高	共享性强，冗余度低且易扩充
数据的独立性	不独立，完全依赖应用程序	独立性弱	具有较强的物理独立性和一定的逻辑独立性
数据的结构化	无结构	记录内有结构、整体无结构	整体结构化，用数据模型描述
数据控制能力	应用程序自己控制	应用程序自己控制	由数据库管理系统提供数据安全性、完整性、并发性控制和数据库恢复功能

文件系统与 DBMS 的区别:
文件系统的数据是面向某一应用的，文件的共享性差、冗余度大、独立性差、整体无结构
DBMS 的数据面向整个组织和企业，共享性高、冗余度小、具有物理和逻辑独立性。

*（1）试述数据管理的文件系统阶段和数据库系统阶段“数据独立性”有何不同。

答：

文件系统中数据被组织成相互独立的数据文件，程序按照文件名访问数据，“数据独立性”是一种“设备独立性”。

数据库系统中的“数据独立性”包括“物理独立性”和“逻辑独立性”，物理独立性是指用户的应用程序与存储在磁盘上的数据库中的数据是相互独立的；逻辑独立性是指用户的应用程序与数据库的逻辑结构是相互独立的。

数据库系统管理数据的特点

- **（1）整体数据的结构化**
- **（2）数据的共享性强，冗余度低且易于扩充**
- **（3）数据的独立性强　数据独立性由数据库管理系统的两级映像功能来保证（物理/逻辑）**
- **（4）数据由数据库管理系统统一管理和控制**

数据库系统最大的特点是三级抽象两级独立性

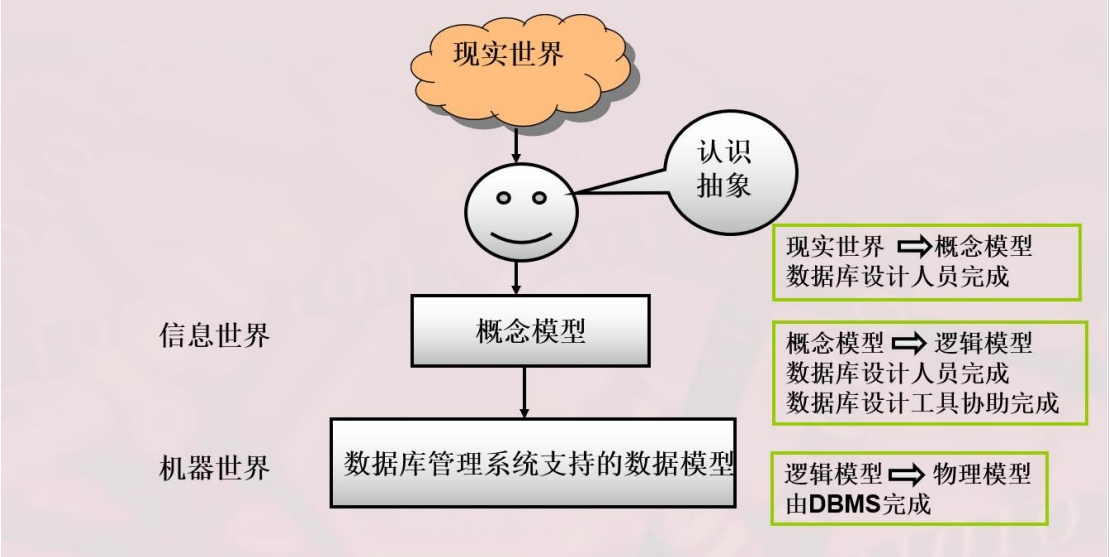
实体型（entity type）

用实体名及其属性名集合来抽象和刻画同类实体称为实体型

实体集（entity set）

同一类型实体的集合称为实体集

- 数据模型分为两类（两个不同的层次）
  - （1） 概念模型　也称信息模型，它是按用户的观点来对数据和信息建模，用于数据库设计。
  - （2） 逻辑模型和物理模型
    - 逻辑模型主要包括网状模型、层次模型、关系模型、面向对象数据模型、对象关系数据模型、半结构化数据模型等。按计算机系统的观点对数据建模，用于**DBMS**实现。
    - 物理模型是对数据最底层的抽象，描述数据在系统内部的表示方式和存取方法，在磁盘或磁带上的存储方式和存取方法。



数据模型概念：数据模型是对现实世界数据特征的抽象，是现实世界的模拟，按计算机系统的观点对数据建模

数据模型作用：数据模型是数据库系统的核心和基础，任何一个 DBMS 都要以一个数据模型为基础，或者支持某个数据模型 数据模型都是逻辑上的

数据模型的三要素

- **1.数据结构　描述数据库的组成对象以及对象之间的联系**
- **2.数据操纵**
- **3.数据约束**

物理模型：数据模型以一定的组织方式存储在数据库管理系统中，是数据模型在数据库管理系统内部的物理存储结构

- **层次模型（树）**
- **网状模型（允许多节点为根，且可以有多个父节点）**
- **关系模型 存取路径对用户隐蔽，用户只要指出“干什么”，不必详细说明“怎么干”**

概念模型：按照用户的观点对数据和信息建模（现实世界到机器世界的一个中间层次，用于信息世界的建模）；最常用的概念模型是 ER 模型

码：可唯一确定一个元组

关系模式（schema）数据库中全体数据的逻辑结构和特征的描述，一般表示为：关系吗（属性、属性）

数据字典是数据库系统中各种描述信息和控制信息的集合

主要内容有：数据项、数据结构、数据流、数据存储、处理过程

外模式：子模式或用户模式，局部数据的逻辑结构和特征的描述，是数据库用户的数据视图

模式：逻辑模式，全体数据的逻辑结构和特征的描述，是所有用户的公共数据视图，外模式通常是内模式的子集

内模式：存储模式，是对数据的物理结构和存储方式的描述

两级映射：保证了物理和逻辑独立性；物理独立性：当物理结构改变，模式和外模式无需更

改；逻辑独立性：当模式修改，外模式无需修改。二者都仅需修改映射

19. 试述数据库管理员、系统分析员、数据库设计人员、应用程序员的职责。

答：

数据库管理员全面负责管理和控制数据库系统。具体职责包括：

① 决定数据库的信息内容和结构；

② 决定数据库的存储结构和存取策略；

③ 定义数据的安全性要求和完整性约束条件；

④ 监控数据库的使用和运行；

⑤ 数据库系统的改进和重组重构。

系统分析员负责应用系统的需求分析和规范说明，他要和用户及 DBA 相结合，确定系统的硬件软件配置，并参与数据库系统的概要设计。

数据库设计人员负责数据库中数据的确定和数据库各级模式的设计。数据库设计人员必须参加用户需求调查和系统分析，然后进行数据库设计。

应用程序员负责设计和编写应用系统的程序模块，并进行调试和安装。

第二章

1. 试述关系模型的三个组成部分。

答：关系模型由关系数据结构、关系操作集合和关系完整性约束三部分组成。

候选码：若关系中的某一属性组的值能唯一地标识一个元组，而其子集不能，则称该属性组为候选码（candidate key）。

主码：若一个关系有多个候选码，则选定其中一个为主码（primary key）。

外码：设 F 是基本关系 R 的一个或一组属性，但不是关系 R 的码，如果 F 与基本关系 S 的主码 K_s 相对应，则称 F 是基本关系 R 的外部码（foreign key），简称外码。

关系是关系模式在某一时刻的状态或内容，关系模式是静态的、稳定的，而关系是动态的，关系是关系模式的一个实例。

在参照完整性中，如果外码属性不是其所在关系的主属性，外码属性的值可以取空值。

- ❖ 常用的关系操作
  - ■ 查询操作：选择、投影、连接、除、并、差、交、笛卡尔积
    - ● 选择、投影、并、差、笛卡尔基是5种基本操作
  - ■ 数据更新：插入、删除、修改
- ❖ 关系操作的特点
  - ■ 集合操作方式：操作的对象和结果都是集合，一次一集合的方式

第三章

1. 试述 SQL 的特点。

答：

① 综合统一。SQL 语言集数据定义语言(DDL)、数据操纵语言(DML)和数据控制语言

(DCL)的功能于一体。

② 高度非过程化。用 SQL 语言进行数据操作，只要提出“做什么”，而无须指明“怎么做”，因此无须了解存取路径。存取路径的选择以及 SQL 语句的操作过程由系统自动完成。

③ 面向集合的操作方式。SQL 语言采用集合操作方式，不仅操作对象、查找结果可以是元组的集合，而且一次插入、删除、更新操作的对象也可以是元组的集合。

④ 以同一种语法结构提供两种使用方式。SQL 语言既是自含式语言，又是嵌入式语言。作为自含式语言，它能够独立地用于联机交互的使用方式，也能够嵌入到高级语言程序中，供程序员设计程序时使用。

⑤ 语言简洁，易学易用。

6. 什么是基本表？什么是视图？两者的区别和联系是什么？

答：

基本表是本身独立存在的表，在 SQL 中一个关系就对应一个基本表。

视图是从一个或几个基本表导出的表。视图本身不独立存储在数据库中，是一个虚表。即数据库中只存放视图的定义而不存放视图对应的数据，这些数据仍存放在导出视图的基本表中。视图在概念上与基本表等同，用户可以如同基本表那样使用视图，可以在视图上再定义视图。

7. 试述视图的优点。

答：

① 视图能够简化用户的操作。

② 视图使用户能以多种角度看待同一数据。

③ 视图对重构数据库提供了一定程度的逻辑独立性。

④ 视图能够对机密数据提供安全保护。

8. 哪类视图是可以更新的，哪类视图是不可更新的？各举一例说明。

答：

基本表的行列子集视图一般是可更新的。如《概论》第 3.7.1 小节中的[例 3.84]。

若视图的属性来自聚集函数、表达式，则该视图肯定是不可以更新的。如《概论》第 3.7.1 小节[例 3.89]中的 S_G 视图。

（3）若一个视图是从单个基本表导出的，并且只是去掉了基本表的某些行和某些列，但保留了主码，这类视图称为_____。

行列子集视图

（4）SQL 语言的数据定义功能包括_____、表定义、视图定义和_____等。

模式定义、索引定义

（1）解释相关子查询和不相关子查询。

答：

在嵌套查询中，如果子查询的查询条件不依赖于父查询，称为不相关子查询；如果子查询的查询条件依赖于父查询，称为相关子查询。

（2）写出 ANY 和 ALL 谓词与聚集函数或 IN 谓词可能存在的等价转换关系。

答：

	=	<>	<	<=	>	>=
ANY	IN	--	< MAX	<=MAX	> MIN	>=MIN
ALL	--	NOT IN	< MIN	<=MIN	> MAX	>=MAX



外连接（续）

[例 3. 53] 改写[例 3.49]

```
SELECT Student.Sno,Sname,Ssex,Sage,Sdept,Cno,Grade
FROM  Student  LEFT OUT JOIN SC ON
        (Student.Sno=SC.Sno);
```

```
SELECT Sname                          /*外层查询/父查询*/
FROM Student
WHERE Sno IN
            ( SELECT Sno              /*内层查询/子查询*/
              FROM SC
              WHERE Cno= ' 2 ');
```

```
SELECT Sno, Cno
FROM SC, (SELECTSno, Avg(Grade)
          FROM SC
          GROUP BY Sno)
          AS   Avg_sc(avg_sno,avg_grade)
WHERE SC.Sno = Avg_sc.avg_sno
 and SC.Grade >=Avg_sc.avg_grade
```

❖如果子查询中没有聚集函数，派生表可以不指定属性列，子查询SELECT子句后面的列名为其缺省属性。

[例3.60]查询所有选修了1号课程的学生姓名，可以用如下查询完成：

```
SELECT Sname
FROM    Student,
        (SELECT Sno FROM SC WHERE Cno=' 1 ') AS SC1
WHERE  Student.Sno=SC1.Sno;
```

```
CREATE VIEW IS_Student
AS
SELECT Sno,Sname,Sage
FROM  Student
WHERE  Sdept= 'IS'
WITH CHECK OPTION;
```

❖若一个视图是从单个基本表导出的，并且只是去掉了基本表的某些行和某些列，但保留了主码，我们称这类视图为行列子集视图。

■ IS_Student视图就是一个行列子集视图。



3.7.4 视图的作用

❖视图能够简化用户的操作

❖视图使用户能以多种角度看待同一数据

❖视图对重构数据库提供了一定程度的逻辑独立性

❖视图能够对机密数据提供安全保护

❖适当的利用视图可以更清晰的表达查询

第四章 数据安全

1. 什么是数据库的安全性？

答：数据库的安全性是指保护数据库以防止不合法的使用所造成的数据泄露、更改或破坏。

4. 试述实现数据库安全性控制的常用方法和技术。

答：

实现数据库安全性控制的常用方法和技术有以下几种。

① 用户身份鉴别：系统提供多种方式让用户标识自己的名字或身份。用户要使用数据库系统时，由系统进行核对，通过鉴定后才可以使用数据库。

② 多层存取控制：系统提供用户权限定义和合法权限检查功能，用户只有获得某种权限才能访问数据库中的某些数据。

③ 视图机制：为不同的用户定义不同的视图，通过视图机制把要保密的数据对无权存取的用户隐藏起来，从而自动对数据提供一定程度的安全保护。

④ 审计：建立审计日志，把用户对数据库的所有操作自动记录下来放入审计日志中，审计员可以利用审计信息重现导致数据库现有状况的一系列事件，找出非法存取数据的人、时间和内容等。

⑤ 数据加密：对存储和传输的数据进行加密处理，从而使不知道解密算法的人无法获知

数据的内容。

❖常用存取控制方法

- ■ 自主存取控制（Discretionary Access Control，简称 DAC）
  - ● C2级
  - ● 用户对不同的数据对象有不同的存取权限
  - ● 不同的用户对同一对象也有不同的权限
  - ● 用户还可将其拥有的存取权限转授给其他用户

❖常用存取控制方法（续）

- ■ 强制存取控制（Mandatory Access Control，简称 MAC）
  - ● B1级
  - ● 每一个数据对象被标以一定的密级
  - ● 每一个用户也被授予某一个级别的许可证
  - ● 对于任意一个对象，只有具有合法许可证的用户才可以存取

4.2.4 授权：授予与回收

1.GRANT

❖ GRANT语句的一般格式：

GRANT <权限>[,<权限>]...

ON <对象类型> <对象名>[,<对象类型> <对象名>]…

TO <用户>[,<用户>]...

[WITH GRANT OPTION];

❖ 语义：将对指定操作对象的指定操作权限授予指定的用户

❖ REVOKE语句的一般格式为：

REVOKE <权限>[,<权限>]...

ON <对象类型> <对象名>[,<对象类型><对象名>]…

FROM <用户>[,<用户>]...[CASCADE | RESTRICT];

REVOKE INSERT

ON TABLE SC

FROM U5 CASCADE ;

■ 将用户U5的INSERT权限收回的时候应该使用CASCADE，否则拒绝执行该语句

[例4.3] 把对表SC的查询权限授予所有用户

```
GRANT SELECT
ON TABLE SC
TO PUBLIC;
```

4.角色权限的收回

REVOKE <权限>[,<权限>]…

ON <对象类型> <对象名>

FROM <角色>[,<角色>]…

- 用户可以回收角色的权限，从而修改角色拥有的权限
- REVOKE执行者是
  - 角色的创建者
  - 拥有在这个（些）角色上的ADMIN OPTION

拥有的权限	可否执行的操作			
	CREATE USER	CREATE SCHEMA	CREATE TABLE	登录数据库，执行数据查询和操纵
DBA	可以	可以	可以	可以
RESOURCE	不可以	不可以	不可以	不可以
CONNECT	不可以	不可以	不可以	可以，但必须拥有相应权限

```
GRANT  <角色1>[,<角色2>]…
TO  <角色3>[,<用户1>]…
[WITH ADMIN OPTION]
```

强制存取控制方法（续）

- 在强制存取控制中，数据库管理系统所管理的全部实体被分为主体和客体两大类
- 主体是系统中的活动实体
  - 数据库管理系统所管理的实际用户
  - 代表用户的各进程
- 客体是系统中的被动实体，受主体操纵
  - 文件、基本表、索引、视图

5. 什么是数据库中的自主存取控制方法和强制存取控制方法？

答：

自主存取控制方法：定义各个用户对不同数据对象的存取权限。当用户对数据库访问时首先检查用户的存取权限。防止不合法用户对数据库的存取。

强制存取控制方法：每一个数据对象被（强制地）标以一定的密级，每一个用户也被（强制地）授予某一个级别的许可证。系统规定只有具有某一许可证级别的用户才能存取某一个密级的数据对象。

自主存取控制中自主的含义是：用户可以将自己拥有的存取权限“自主”地授予别人。即用户具有一定的“自主”权。

11. 什么是数据库的审计功能，为什么要提供审计功能？

答：

审计功能是指 DBMS 的审计模块在用户对数据库执行操作的同时，把所有操作自动记录到系统的审计日志中。

因为任何系统的安全保护措施都不是完美无缺的，蓄意盗窃破坏数据的人总可能存在。利用数据库的审计功能，审计员可以根据审计日志中记录的信息，分析和重现导致数据库现有状况的一系列事件，找出非法存取数据的人、时间和内容等。

（3）数据库角色实际上是一组与数据库操作相关的各种 权限 。

第五章 数据库完整性

1. 什么是数据库的完整性？

答：数据库的完整性是指数据的正确性和相容性。

❖数据库的完整性

- ■ 数据的正确性
  - ● 是指数据是符合现实世界语义，反映了当前实际状况的
- ■ 数据的相容性
  - ● 是指数据库同一对象在不同关系表中的数据是符合逻辑的

❖数据的完整性和安全性是两个不同概念

- ■ 数据的完整性
  - ● 防止数据库中存在不符合语义的数据，也就是防止数据库中存在不正确的数据
  - ● 防范对象：不合语义的、不正确的数据
- ■ 数据的安全性
  - ● 保护数据库 防止恶意的破坏和非法的存取
  - ● 防范对象：非法用户和非法操作

```
CREATE TABLE SC
( Sno   CHAR(9)  NOT NULL，
  Cno   CHAR(4)  NOT NULL，
  Grade  SMALLINT,
  PRIMARY KEY(Sno,Cno)，
  FOREIGN KEY (Sno) REFERENCES Student(Sno)
    ON DELETE CASCADE        /*级联删除SC表中相应的元组*/
    ON UPDATE CASCADE,       /*级联更新SC表中相应的元组*/
  FOREIGN KEY (Cno) REFERENCES Course(Cno)
    ON DELETE NO ACTION
      /*当删除course 表中的元组造成了与SC表不一致时拒绝删除*/
    ON UPDATE CASCADE
      /*当更新course表中的cno时，级联更新SC表中相应的元组*/
 );
```

Grade SMALLINT CHECK (Grade>=0 AND Grade <=100)，
/*Grade取值范围是0到100*/

❖ CREATE TRIGGER语法格式

CREATE TRIGGER <触发器名>
{BEFORE | AFTER} <触发事件> ON <表名>
REFERENCING NEW|OLD ROW AS<变量>
FOR EACH {ROW | STATEMENT}
[WHEN <触发条件>]<触发动作体>

```
CREATE TRIGGER  SC_T
AFTER UPDATE OF Grade ON SC
 REFERENCING
      OLD row  AS  OldTuple,
      NEW row AS  NewTuple
FOR EACH ROW
WHEN (NewTuple.Grade >= 1.1*OldTuple.Grade)
   INSERT INTO SC_U(Sno,Cno,OldGrade,NewGrade)
 VALUES(OldTuple.Sno,OldTuple.Cno,OldTuple.Grade,NewTuple.Gr
    ade)
```

被参照表(例如 Student)		参照表(例如 SC)	违约处理
可能破坏参照完整性	←	插入元组	拒绝
可能破坏参照完整性	←	修改外码值	拒绝
删除元组	→	可能破坏参照完整性	拒绝/级联删除/设置为空值
修改主码值	→	可能破坏参照完整性	拒绝/级联修改/设置为空值

7. 在关系系统中，当操作违反实体完整性、参照完整性和用户定义的完整性约束条件时，一般是如何分别进行处理的？

答：对于违反实体完整性和用户定义的完整性的操作，一般都采用拒绝执行的方式进行处理；而对于违反参照完整性的操作，并不都是简单地拒绝执行，有时要根据应用语义执行一些附加的操作，以保证数据库的正确性。具体的处理可以参见上面第 5 题或《概论》第 5.2.2 小节。

3. 什么是数据库的完整性约束条件？

答：完整性约束条件是指数据库中的数据应该满足的语义约束条件。

4. DBMS 的完整性控制机制应具有哪三个方面的功能?

答:

① 定义功能,即提供定义完整性约束条件的机制。

② 检查功能,即检查用户发出的操作请求是否违背了完整性约束条件。

③ 违约处理功能:如果发现用户的操作请求使数据违背了完整性约束条件,则采取一定的动作来保证数据的完整性。

```
CONSTRAINT C1 CHECK(Age<=60),
CONSTRAINT FK_DEPTNO FOREIGN KEY(Deptno)REFERENCES DEPT(Deptno));
```

① ALTER TABLE 项目 ADD CONSTRAINT C1 FOREIGN KEY(负责人编号) REFERENCES 研究人员(人员编号) ON UPDATE NO ACTION ; /*违反约束时 KingbaseES 的默认值是 NO ACTION,所以 NO ACTION 也可以缺省*/

② ALTER TABLE 项目 ADD CONSTRAINT C2 FOREIGN KEY(负责人编号) REFERENCES 研究人员(人员编号) ON DELETE SET NULL ON UPDATE SET NULL;

③ ALTER TABLE 项目 ADD CONSTRAINT c3 FOREIGN KEY(负责人编号) REFERENCES 研究人员(人员编号) ON DELETE CASCADE ON UPDATE CASCADE;

④ ALTER TABLE 参与 ADD CONSTRAINT c4 CHECK(工作时间 >=1 AND 工作时间 <=12);

⑤ ALTER TABLE 项目 ADD CONSTRAINT c5 CHECK(名称 IS NOT NULL);

(4)

①
```
CREATE TRIGGER NewTeacher
AFTER INSERT ON Teacher
FOR EACH ROW AS
BEGIN  /*插入新教师后将此教师信息插入到 Work 中,不确定的属性赋以 NULL*/
    INSERT INTO work VALUES(new.tno,NULL,NULL,NULL);
END;
```

②
```
CREATE TRIGGER UpdateAge
AFTER UPDATE of tage on Teacher
FOR EACH ROW AS
BEGIN
    IF(new.tage < old.tage)      /*更新教师年龄时,如果新年龄比旧年龄小*/
        UPDATE Teacher SET tage=old.tage WHERE tno=new.tno;  /*则用旧年龄代替*/
END;
```

注意:不同的 RDBMS 实现的触发器语法各不相同,互不兼容。请读者在上机实验时注意阅读所用系统的使用说明。

第六章 关系代数理论

范式通俗理解: 1NF、2NF、3NF 和 BNCF-CSDN 博客

完全函数依赖、部分函数依赖：在 $R(U)$ 中，如果 $X \rightarrow Y$，并且对于 X 的任何一个真子集 X'，都有 $X' \not\rightarrow Y$，则称 Y 对 X 完全函数依赖，记作：

$$X \xrightarrow{F} Y$$

若 $X \rightarrow Y$，但 Y 不完全函数依赖于 X，则称 Y 对 X 部分函数依赖，记作：

$$X \xrightarrow{P} Y$$

传递依赖：在 $R(U)$ 中，如果 $X \rightarrow Y$，$Y \not\subseteq X$，$Y \not\rightarrow X$，$Y \rightarrow Z$，$Z \not\subseteq Y$，则称 Z 对 X 传递函数依赖。

候选码、主码：设 K 为 $R<U,F>$ 中的属性或属性组合，若 $K \xrightarrow{F} U$，则 K 为 R 的候选码。若候选码多于一个，则选定其中的一个为主码。

外码：关系模式 R 中属性或属性组 X 并非 R 的码，但 X 是另一个关系模式的码，则称 X 是 R 的外部码，也称外码。

全码：整个属性组是码，称为全码(all-key)。

1NF：如果一个关系模式 R 的所有属性都是不可分的基本数据项，则 $R \in 1NF$。

＊解析：

第一范式是对关系模式的最起码的要求。不满足第一范式的数据库模式不能称为关系数据库。

答：

2NF：若关系模式 $R \in 1NF$，并且每一个非主属性都完全函数依赖于 R 的码，则 $R \in 2NF$。

3NF：关系模式 $R<U,F>$ 中若不存在这样的码 X，属性组 Y 及非主属性 $Z(Z \not\subseteq Y)$ 使得 $X \rightarrow Y$，$(Y \not\rightarrow X)$，$Y \rightarrow Z$ 成立，则称 $R<U,F> \in 3NF$。

BCNF：关系模式 $R<U,F> \in 1NF$。若 $X \rightarrow Y$ 且 $Y \not\subseteq X$ 时 X 必含有码，则 $R<U,F> \in BCNF$。

＊解析：

读者要真正理解这些范式的内涵。各种范式之间的联系：$5NF \subset 4NF \subset BCNF \subset 3NF \subset 2NF \subset 1NF$(《概论》图 6.2)。能够理解为什么有这种包含关系。

答：

多值依赖：设 $R(U)$ 是属性集 U 上的一个关系模式，X、Y、Z 是 U 的子集，并且 $Z=U-X-Y$。关系模式 $R(U)$ 中多值依赖 $X \rightarrow\rightarrow Y$ 成立，当且仅当对 $R(U)$ 的任一关系 r，给定的一对 (x,z) 值，有一组 Y 的值，这组值仅仅决定于 x 值而与 z 值无关。

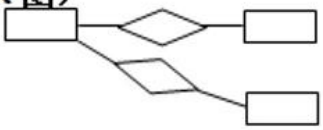
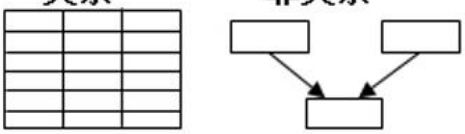
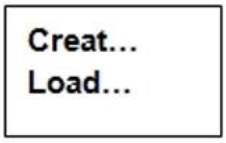
4NF：关系模式 $R<U,F> \in 1NF$，如果对于 R 的每个非平凡多值依赖 $X \rightarrow\rightarrow Y(Y \not\subseteq X)$，$X$ 都含有码，则称 $R<U,F> \in 4NF$。

分解的原则:

1．无损链接 2.保持函数依赖不变

第七章 数据库设计

- 需求分析和概念设计独立于任何数据库管理系统
- 逻辑设计和物理设计与选用的数据库管理系统密切相关

设计阶段	设计描述
需求分析	数字字典、全系统中数据项、数据结构、数据流、数据存储的描述
概念结构设计	概念模型（E-R 图）  数据字典
逻辑结构设计	某种数据模型 关系 非关系 
物理结构设计	存储安排 存取方法选择 存取路径建立 
数据库实施	创建数据库模式 装入数据 数据库试运行 
数据库运行和维护	性能监测、转储/恢复、数据库重组和重构

需求分析中的数据字典主要用于定义和描述系统中各类数据的业务含义、格式、有效范围等，侧重于从业务角度确保数据的一致性和清晰性。而关系数据库中的数据字典则是数据库管理系统存储的数据结构信息，描述了数据库中表、列、索引、约束等对象的定义和属性，侧重于技术实现和数据库管理。二者的联系在于，需求分析阶段的数据字典为数据库设计提供了基础，确保数据库中的数据结构和业务需求的一致性。

1. 试述数据库设计过程。

这里只概要列出数据库设计过程的 6 个阶段：

① 需求分析；

② 概念结构设计；

③ 逻辑结构设计；

④ 数据库物理设计；

⑤ 数据库实施；

⑥ 数据库运行和维护。

2. 试述数据库设计过程中形成的数据库模式。

答：

数据库设计的不同阶段形成数据库的各级模式，即：

① 在概念结构设计阶段形成独立于机器特点、独立于各个 DBMS 产品的概念模式，在本篇中就是E-R 图。

② 在逻辑结构设计阶段将 E-R 图转换成具体的数据库产品支持的数据模型，如关系模型，形成数据库逻辑模式；然后在基本表的基础上再建立必要的视图(view)，形成数据的外模式。

③ 在物理结构设计阶段，根据 DBMS 特点和处理的需要进行物理存储安排，建立索引，形成数据库内模式。

4. 数据字典的内容和作用是什么？

答：

数据字典的内容通常包括数据项、数据结构、数据流、数据存储和处理过程 5 个部分。其中数据项是数据的最小组成单位，若干个数据项可以组成一个数据结构。数据字典通过对数据项和数据结构的定义来描述数据流、数据存储的逻辑内容。

数据字典的作用：数据字典是关于数据库中数据的描述，在需求分析阶段建立，是下一步进行概念设计的基础，并在数据库设计过程中不断修改、充实和完善。

9. 什么是数据库的逻辑结构设计？试述其设计步骤。

答：

数据库的逻辑结构设计就是把概念结构设计阶段设计好的基本 E-R 图转换为与选用的 DBMS 产品所支持的数据模型相符合的逻辑结构。设计步骤为：

① 将概念结构转换为关系模型；

② 对数据模型进行优化。

13. 试述数据库物理设计的内容和步骤。

答：

数据库在物理设备上的存储结构与存取方法称为数据库的物理结构，它依赖于给定的 DBMS。为一个给定的逻辑数据模型选取一个最适合应用要求的物理结构，就是数据库的物理设计的主要内容。

数据库的物理设计步骤通常分为两步：

① 确定数据库的物理结构，在关系数据库中主要指存取方法和存储结构；

② 对物理结构进行评价，评价的重点是时间和空间效率。

15. 什么是数据库的再组织和重构造？为什么要进行数据库的再组织和重构造？

答：

数据库的再组织是指按原设计要求重新安排存储位置、回收垃圾、减少指针链等，以提高系统性能。

数据库的重构造则是指部分修改数据库的模式和内模式，即修改原设计的逻辑和物理结构。数据库的再组织是不修改数据库的模式和内模式的。

进行数据库的再组织和重构造的原因：

数据库运行一段时间后，由于记录不断增、删、改，会使数据库的物理存储情况变坏，降低了数据的存取效率，数据库性能下降，这时DBA 就要对数据库进行重组织。DBMS 一般都提供数据重组织用的实用程序。

数据库应用环境常常发生变化，如增加新的应用或新的实体，取消了某些应用，有的实体与实体间的联系也发生了变化等，使原有的数据库设计不能满足新的需求，需要调整数据库的模式和内模式。这就要进行数据库重构造。

数据库常见存取方式：索引、聚簇、hash

数据字典：数据项、数据结构、数据流、数据存储、处理过程

第八章 数据库编程

EXEC SQL DECLARE <游标名> CURSOR FOR <SELECT语句>;

Open 、 fetch、close

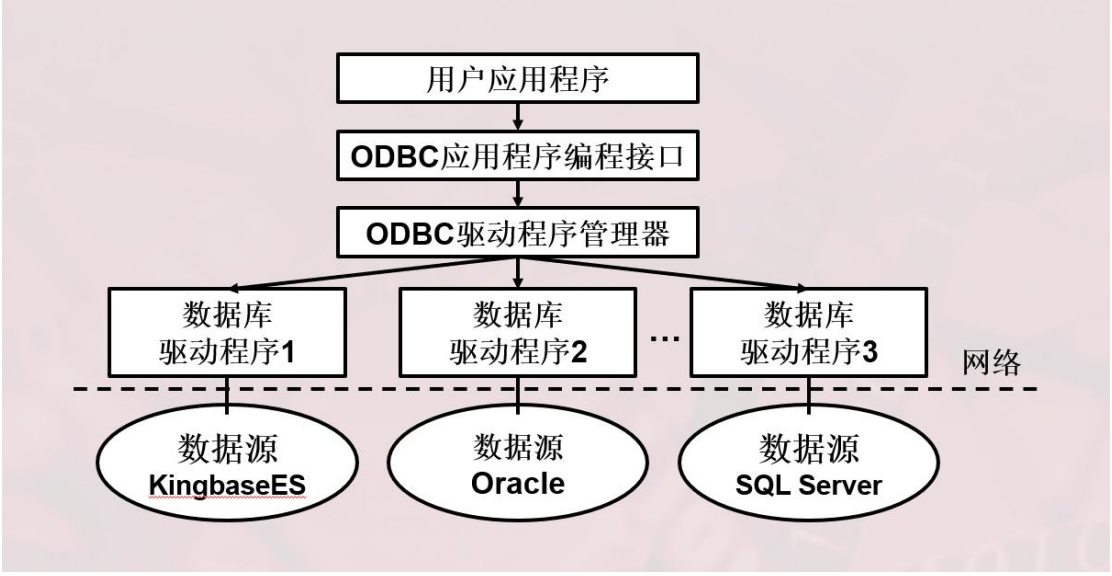
CREATE OR REPLACE FUNCTION 函数名 ([参数1,参数2,...]) RETURNS <类型> AS <过程化SQL块>;

Call/select

（1）创建存储过程

CREATE OR REPLACE PROCEDURE 过程名([参数1,参数2,...]) AS <过程化SQL块>;

Call/perform



ODBC 的全称是 Open Database Connectivity，即 开放数据库连接。

JDBC 的全称是 Java Database Connectivity，即 Java数据库连接。

第九章 查询优化

❖关系数据库管理系统查询处理阶段：

1. 查询分析
2. 查询检查
3. 查询优化
4. 查询执行

6. 试述关系数据库管理系统查询优化的一般准则。

答：

下面的优化策略一般能提高查询效率：

① 选择运算应尽可能先做。

② 把投影运算和选择运算同时进行。

③ 把投影同其前或其后的双目运算结合起来执行。

④ 把某些选择同在它前面要执行的笛卡儿积结合起来成为一个连接运算。

⑤ 找出公共子表达式。

⑥ 选取合适的连接算法。

解析：

①~⑤是指代数优化策略。⑥涉及物理优化。

第十章 数据库恢复

❖ 恢复操作的基本原理：冗余

1. 试述事务的概念及事务的 4 个特性。恢复技术能保证事务的哪些特性?

答:

事务是用户定义的一个数据库操作序列,这些操作要么全做、要么全不做,是一个不可分割的工作单位。

事务具有 4 个特性:原子性(Atomicity)、一致性(Consistency)、隔离性(Isolation)和持续性(Durability)。这 4 个特性也简称为ACID 特性。

原子性:事务是数据库的逻辑工作单位,事务中包括的诸操作要么都做,要么都不做。

一致性:事务执行的结果必须是使数据库从一个一致性状态变到另一个一致性状态。

隔离性:一个事务的执行不能被其他事务干扰。即一个事务内部的操作及使用的数据对其他并发事务是隔离的,并发执行的各个事务之间不能互相干扰。

持续性:持续性也称永久性(permanence),指一个事务一旦提交,它对数据库中数据的改变就应该是永久性的。接下来的其他操作或故障不应该对其执行结果有任何影响。

故障恢复可以保证事务的原子性与持续性。

2. 为什么事务非正常结束时会影响数据库数据的正确性,请举例说明之。

答:

事务执行的结果必须是使数据库从一个一致性状态变到另一个一致性状态。如果数据库系统运行中发生故障,有些事务尚未完成就被迫中断,这些未完成事务对数据库所做的修改有一部分已写入物理数据库,这时数据库就可能处于不正确的状态,或者说是不一致的状态。

例如某工厂的库存管理系统中,要把数量为 Q 的某种零件从仓库 1 移到仓库 2 存放,则可以定义一个事务 T,T 包括两个操作:$Q1=Q1-Q$,$Q2=Q2+Q$。如果 T 非正常终止时只做了第一个操作,则数据库就处于不一致性状态,库存量无缘无故少了 Q。

6. 针对不同的故障,试给出恢复的策略和方法。(即如何进行事务故障的恢复、系统故障的恢复,以及如何进行介质故障恢复。)

答:

事务故障的恢复步骤是:

① 反向扫描文件日志,查找该事务的更新操作。

② 对该事务的更新操作执行逆操作。即将日志记录中"更新前的值"写入数据库。

直至读到此事务的开始标记,该事务故障的恢复就完成了。

系统故障的恢复步骤是:

① 正向扫描日志文件,找出在故障发生前已经提交的事务队列(REDO 队列)和未完成的事务队列(UNDO 队列)。

② 对未完成的事务队列中的各个事务进行 UNDO 处理。

③ 对已经提交的事务队列中的各个事务进行 REDO 处理。

介质故障的恢复步骤是:

① 装入最新的数据库后备副本(离故障发生时刻最近的转储副本),使数据库恢复到最近一次转储时的一致性状态。

② 装入转储结束时刻的日志文件副本。

③ 启动系统恢复命令,由 DBMS 完成恢复功能,即重做已完成的事务。

9. 试述使用检查点方法进行恢复的步骤。

答：

① 在重新开始文件(见第 7 题的图)中，找到最后一个检查点记录在日志文件中的地址，由该地址在日志文件中找到最后一个检查点记录。

② 由该检查点记录得到检查点建立时刻所有正在执行的事务清单 ACTIVE-LIST。

这里建立两个事务队列：

- UNDO-LIST：需要执行 undo 操作的事务集合；
- REDO-LIST：需要执行 redo 操作的事务集合。

把 ACTIVE-LIST 暂时放入 UNDO-LIST 队列，REDO 队列暂为空。

③ 从检查点开始正向扫描日志文件

- 如有新开始的事务 T_i，把 T_i 暂时放入 UNDO-LIST 队列；

- 如有提交的事务 T_j，把 T_j 从 UNDO-LIST 队列移到 REDO-LIST 队列，直到日志文件结束。

④ 对 UNDO-LIST 中的每个事务执行 UNDO 操作，对 REDO-LIST 中的每个事务执行 REDO 操作。

(1) 在系统故障的恢复策略中，为什么 UNDO 处理反向扫描日志文件，REDO 处理正向扫描日志文件。

答：

如果存在同一个数据的多个 UNDO 操作，需要将数据恢复到第一个失败事务之前，如果正向扫描处理日志文件，无法实现这一目标，因此应该反向扫描日志文件。对于同一个数据的多个 REDO 操作，需要将数据恢复到最后一个成功事务之后，因此应该正向扫描日志文件。

(2) 说明恢复系统是否可以保证事务的原子性和持续性。

答：

原子性是指操作要么都做，要么都不做，在恢复策略中 UNDO 可以保证将未成功提交的事务所有操作都取消，REDO 可以保证将成功提交的事务所有操作都完成，因此需确保事务的原子性；持续性是指一旦事务提交，对数据库中数据的改变是永久性的，REDO 可以保证事务只要提交，改变一定被永久实现，因此要确保事务的持续性。

第十一章 并发控制

并发控制可以保证事务的一致性和隔离性。

2. 并发操作可能会产生哪几类数据不一致？用什么方法能避免各种不一致的情况？

答：

并发操作带来的数据不一致性包括三类：

（1）丢失修改

两个事务 T_1 和 T_2 读入同一数据并修改，T_2 提交的结果破坏了（覆盖了）T_1 提交的结果，导致 T_1 的修改被丢失。

（2）不可重复读

不可重复读是指事务 T_1 读取某一数据后，事务 T_2 对其执行更新操作，使 T_1 无法再现前一次读取结果。不可重复读包括三种情况：

① 事务 T_1 读取某一数据后，事务 T_2 对其做了修改，当事务 T_1 再次读该数据时，得到与前一次不同的值。

② 事务 T_1 按一定条件从数据库中读取了某些数据记录后，事务 T_2 删除了其中部分记录，当 T_1 再次按相同条件读取数据时，发现某些记录消失了。

③ 事务 T_1 按一定条件从数据库中读取某些数据记录后，事务 T_2 插入了一些记录，当 T_1 再次按相同条件读取数据时，发现多了一些记录。

后两种不可重复读有时也称为幻影（phantom row）现象。

（3）读“脏”数据

读“脏”数据是指事务 T_1 修改某一数据，并将其写回磁盘，事务 T_2 读取同一数据后，T_1 由于某种原因被撤销，这时 T_1 已修改过的数据恢复原值，T_2 读到的数据就与数据库中的数据不一致，则 T_2 读到的数据就为“脏”数据，即不正确的数据。

避免不一致性的方法就是并发控制。常用的并发控制技术包括封锁方法、时间戳方法、乐观控制方法和多版本并发控制方法等。

防止死锁的发生其实就是要破坏产生死锁的条件。预防死锁通常有两种方法：

① 一次封锁法

要求每个事务必须一次将所有要使用的数据全部加锁，否则就不能继续执行。

② 顺序封锁法

预先对数据对象规定一个封锁顺序，所有事务都按这个顺序实行封锁。

7. 请给出检测死锁发生的一种方法，当发生死锁后如何解除死锁？

答：

数据库系统一般采用的方法是允许死锁发生，DBMS 检测到死锁后加以解除。

DBMS 中诊断死锁的方法与操作系统类似，一般使用超时法或事务等待图法。

超时法是指如果一个事务的等待时间超过了规定的时限，就认为发生了死锁。

DBMS 并发控制子系统检测到死锁后，就要设法解除。通常采用的方法是选择一个处理死锁代价最小的事务，将其撤销，释放此事务持有的所有锁，使其他事务得以继续运行下去。

	X锁		S锁		一致性保证			隔离性级别保证
	操作结束释放	事务结束释放	操作结束释放	事务结束释放	不丢失修改	不“脏”读	可重复读	
一级封锁协议		√			√			读未提交
二级封锁协议		√	√		√	√		读已提交
三级封锁协议		√		√	√	√	√	可串行化

❖死锁的诊断

（1）超时法

（2）事务等待图法

❖两段锁的含义

事务分为两个阶段

- 第一阶段是获得封锁，也称为扩展阶段
  - 事务可以申请获得任何数据项上的任何类型的锁，但是不能释放任何锁
- 第二阶段是释放封锁，也称为收缩阶段
  - 事务可以释放任何数据项上的任何类型的锁，但是不能再申请任何锁

8. 什么样的并发调度是正确的调度？

答：

可串行化的调度是正确的调度。

可串行化的调度的定义：多个事务的并发执行是正确的，当且仅当其结果与按某一次序串行地执行它们时的结果相同，称这种调度策略为可串行化的调度。

S